COMPILADORES

AULA 17

Ambientes de execução e geração de código

Memória, registros de ativação, código de destino e máquina de pilha

Material didático — Projeto MINIC

Aulas de 07/10 e 08/10 | Capítulos 7 (seções 7.1–7.3) e 8 (seções 8.1–8.3)

Como esta aula se encaixa no compilador

Nas aulas anteriores, o front-end do MINIC reconheceu tokens, construiu a AST, vinculou nomes a símbolos, verificou tipos e começou a traduzir construções para uma representação intermediária (IR), recomendada como código de três endereços. Nesta aula, estudamos como os valores e as chamadas podem existir durante a execução e como a IR pode ser mapeada para uma máquina-alvo.

A ordem conceitual é importante: a análise semântica determina o significado e os tipos; a IR explicita operações e fluxo de controle; o ambiente de execução oferece lugares e mecanismos para valores, ativações e chamadas; o gerador de código escolhe instruções que preservam esse significado. A otimização em blocos e entre blocos será estudada nas aulas seguintes, portanto aqui a prioridade é gerar código correto e compreensível.

Objetivos de aprendizagem

- Distinguir alocação estática, pilha de execução e heap pelo tempo de vida e pela forma de acesso aos dados.
- Explicar ativação, registro de ativação, endereço de retorno, parâmetros, variáveis locais e recursão.
- Relacionar escopo léxico e acesso a nomes não locais, sem confundir escopo com tempo de vida.
- Descrever as responsabilidades de um gerador de código e a seleção de instruções para uma máquina-alvo.
- Traduzir expressões, atribuições, decisões, laços e chamadas para IR e para uma máquina de pilha didática.

Vocabulário essencial

Termo	Definição operacional
Ativação	Uma execução específica de uma função, desde sua entrada até seu retorno. Chamadas recursivas distintas são ativações distintas.
Registro de ativação (frame)	Estrutura de dados que guarda informações necessárias a uma ativação: parâmetros, estado salvo, dados locais e outros campos definidos pela convenção de chamada.
Ambiente de execução	Conjunto de áreas de memória, convenções e rotinas que sustentam a execução do programa objeto.
Código de destino	Programa produzido para a máquina-alvo escolhida: por exemplo, assembly, código de máquina ou bytecode de uma VM.
Convenção de chamada	Contrato entre chamador e chamado sobre argumentos, resultado, registradores preservados, pilha e retorno.

1. Organização de memória e tempo de vida

O programa em execução ocupa um espaço de endereçamento lógico. O sistema operacional e a arquitetura relacionam esse espaço à memória física; o compilador define ou pressupõe leiautes e formas de acesso compatíveis com a máquina-alvo. Um modelo didático separa código, dados estáticos, pilha e heap. Essa divisão é conceitual: sistemas reais podem organizar regiões de outras maneiras.

Região	Uso típico	Tempo de vida / observação
Código	Instruções executáveis e constantes associadas.	Em geral, permanece disponível durante a execução do programa.
Área estática	Variáveis globais e dados cujo endereço pode ser fixado antes da execução.	Normalmente cobre toda a execução do programa; não é sinônimo de “memória rápida”.
Pilha de execução	Registros de ativações de funções e dados de vida limitada à chamada.	Aloca e libera de modo compatível com chamadas e retornos aninhados.
Heap (monte)	Objetos cuja criação ou duração não se ajusta naturalmente a uma única ativação.	Alocação/liberação dinâmica; pode exigir gerenciamento explícito ou coleta, conforme a linguagem.

Estático não significa “toda variável local”

“Estático” descreve uma decisão de alocação/endereço que pode ser determinada antes da execução. Variáveis globais do MINIC são um exemplo natural. Variáveis locais ordinárias são associadas a uma ativação e, portanto, normalmente ocupam a pilha. O fato de o compilador conhecer seu tamanho não transforma seu tempo de vida em estático.

Na especificação mínima fornecida, MINIC define globais, locais, parâmetros, blocos e vetores de tamanho declarado. Ela não define malloc/free nem operadores de ponteiro. Assim, heap é apresentado como conceito geral de ambiente de execução, não como recurso que o projeto MINIC esteja obrigado a implementar. Uma implementação pode usar heap internamente, mas isso não cria automaticamente uma operação de heap visível na linguagem.

Exemplo: vida de uma variável local

```
int somaAte(int n) {
     int acumulado = 0;
     int i;
     for (i = 1; i <= n; i = i + 1) {
         acumulado = acumulado + i;
     }
     return acumulado;
}
```

Em uma execução de somaAte, n, acumulado e i pertencem àquela ativação. Ao retornar, seus espaços de pilha deixam de representar variáveis vivas dessa chamada; uma chamada posterior pode reutilizar os mesmos endereços. Isso não implica que os bits sejam apagados, apenas que o armazenamento não deve mais ser acessado como aquelas variáveis.

2. Ativações, pilha e registros de ativação

Uma chamada cria uma ativação do procedimento chamado. Como as chamadas usuais terminam em ordem inversa à sua abertura, a pilha é uma estrutura adequada: a ativação corrente fica no topo; ao retornar, seu registro é removido e o controle continua na ativação chamadora. A sequência de chamadas forma um caminho ativo em uma árvore de ativações.

Campos possíveis de um registro de ativação

O leiaute exato depende da linguagem, arquitetura, compilador e convenção de chamada. Alguns campos podem ficar em registradores, não na memória. A lista abaixo é um modelo explicativo, não um leiaute universal.

Campo	Para que serve
Parâmetros / argumentos	Disponibilizam ao chamado os valores recebidos; podem chegar por registradores, pilha ou ambos.
Valor de retorno	Comunica o resultado ao chamador, muitas vezes por registrador dedicado.
Endereço de retorno	Ponto de continuação no chamador após a chamada. Pode ser salvo pela instrução de chamada ou pelo prólogo.
Elo de controle (dinâmico)	Referencia o frame do chamador; ajuda a restaurar a cadeia de chamadas.
Registradores salvos / estado	Preserva valores exigidos pela convenção para que o chamador continue corretamente.
Variáveis locais e temporários	Armazenam dados da ativação que não permanecem em registradores.
Elo de acesso (quando necessário)	Permite localizar dados de escopos léxicos envolventes em linguagens com procedimentos aninhados.

Recursão exige um frame por chamada ativa

```
int fatorial(int n) {
     if (n <= 1) return 1;
     return n * fatorial(n - 1);
}
```

Ao avaliar fatorial(3), a execução abre ativações fatorial(3), fatorial(2) e fatorial(1). Cada uma precisa preservar seu próprio n e o contexto para continuar a multiplicação após o retorno da chamada interna. Um único espaço global para n seria incorreto: a chamada mais profunda sobrescreveria os valores das chamadas anteriores.

```
Topo da pilha  →  fatorial(1): n=1, retorno para fatorial(2)
                  fatorial(2): n=2, retorno para fatorial(3)
Fundo da pilha →  fatorial(3): n=3, retorno para main
```

A chamada recursiva não “reinicia a função”: cria outra ativação da mesma função. Quando fatorial(1) retorna 1, o controle restaura a ativação fatorial(2), calcula 2 × 1; em seguida restaura fatorial(3) e calcula 3 × 2.

Uma sequência conceitual de chamada e retorno

1. O chamador avalia os argumentos segundo a ordem definida para a implementação e os disponibiliza ao chamado.
2. A operação de chamada transfere o controle e registra como retornar; o prólogo estabelece o frame e salva o estado necessário.
3. O corpo executa usando parâmetros e locais da ativação corrente.
4. O retorno coloca o resultado no local convencionado, restaura o estado e transfere o controle ao endereço de retorno.
5. O chamador continua após a chamada e consome o resultado, se houver.

O compilador pode dividir essas tarefas entre chamador e chamado. Não existe uma divisão universal: o contrato precisa ser consistente entre quem emite a chamada e quem implementa a função.

3. Variáveis não locais e escopo

Escopo é uma propriedade estática da linguagem: determina a qual declaração um uso de nome se refere. Tempo de vida é uma propriedade da execução: determina durante quanto tempo o objeto permanece válido. A tabela de símbolos resolve o vínculo entre nome e declaração; o gerador/ambiente escolhe como localizar o valor em execução.

```
int limite = 10;  // global
int conta(int x) {
     int resultado;  // local à ativação de conta
     resultado = x + limite;
     return resultado;
}
```

Em conta, x e resultado são locais à ativação corrente; limite é não local em relação à função e, no modelo MINIC sem funções aninhadas, pode ser acessado por endereço estático conhecido pelo compilador. Blocos aninhados podem introduzir locais com duração restrita ao bloco; a implementação pode alocar slots no frame e reutilizá-los quando os tempos de vida não se sobrepõem.

Elo de controle e elo de acesso não são a mesma coisa. O elo de controle aponta para quem chamou a ativação; um elo de acesso, quando a linguagem o exige, segue a estrutura léxica dos procedimentos declarados dentro de outros. A especificação MINIC apresentada não permite declarar função dentro de função; por isso, elos de acesso entre frames de funções aninhadas não são requisito mínimo. Variáveis globais e escopos de blocos continuam sendo tratados normalmente.

4. Do código intermediário ao código de destino

A entrada do gerador de código é a IR já construída e validada pelo front-end, acompanhada de informações da tabela de símbolos necessárias para endereços, tipos e armazenamento. O gerador não deve reinterpretar erros de sintaxe ou tipos como se fossem válidos: a premissa é receber uma IR consistente, com conversões explícitas quando a semântica as exige.

AST e IR servem a propósitos diferentes. A AST preserva a estrutura abstrata do programa-fonte; uma IR como 3AC explicita operações elementares, temporários e saltos. Código de destino é específico a uma máquina ou VM. A tradução deve preservar o comportamento observável do programa, inclusive ordem e efeitos relevantes.

Representação	Exemplo para x = a + b * 2;	Papel
AST	Assign(x, Add(a, Mul(b, 2)))	Estrutura hierárquica e significado sintático.
3AC	t1 = b * 2 t2 = a + t1 x = t2	Operações simples; explícita dependências e valores temporários.
Código de destino	LOAD a; LOAD b; PUSH 2; MUL; ADD; STORE x	Instruções executáveis conforme a semântica da máquina-alvo.

Gerar código correto antes de gerar código rápido

Um gerador pode usar modelos simples por instrução e ainda assim produzir código correto, embora redundante. Qualidade envolve, entre outros fatores, número de instruções, acessos à memória, pressão sobre registradores e latência. Nesta etapa, a regra de ouro é preservar o significado; melhorias locais e globais serão estudadas nas aulas de otimização.

5. Seleção de instruções, registradores e ordem de avaliação

Seleção de instruções

Selecionar instruções é mapear cada operação da IR a uma sequência disponível na máquina-alvo. A estratégia depende do nível de abstração da IR, do conjunto de instruções e dos custos relativos. Uma instrução de alto nível pode exigir várias instruções simples; uma instrução especializada pode executar a mesma operação com menos trabalho. O mapeamento deve respeitar tipos e efeitos da operação.

Registradores: alocação e atribuição

Registradores são armazenamento rápido, mas finito. Alocação decide quais valores permanecem em registradores em cada ponto; atribuição escolhe quais registradores concretos os guardam. Valores que não cabem podem ser “derramados” (spilled) para slots de memória. A convenção de chamada também importa: alguns registradores podem ser temporários e outros devem ser preservados pelo chamador ou chamado.

```
IR:  t1 = a + b
     t2 = t1 * c
     x  = t2
Alvo de registradores (esquemático):
      LOAD R0, a
      ADD  R0, R0, b
     ; R0 contém t1
      MUL  R0, R0, c
     ; t1 já não é necessário; R0 reutilizado
     STORE x, R0
```

Este exemplo pressupõe uma máquina com instruções de três operandos no formato indicado e operandos de memória permitidos para b e c. Não é assembly universal; serve para ilustrar a correspondência IR–alvo. Se a máquina não permitir esse modo, o gerador precisa carregar cada operando em registrador.

Ordem de avaliação

A ordem de avaliação pode alterar quantos temporários precisam ficar vivos e, portanto, quantos registradores são necessários. Para uma expressão aritmética sem efeitos colaterais, algumas ordens podem produzir o mesmo valor, mas diferir em custo. Não se deve reordenar operações quando isso muda o resultado, o comportamento de curto-circuito, uma leitura/atribuição observável ou um erro de execução.

A especificação MINIC define precedência e associatividade dos operadores, mas não estabelece em detalhe uma ordem geral de avaliação dos argumentos nem a semântica de curto-circuito para && e ||. A implementação precisa documentar e aplicar uma decisão consistente; não deve apresentar uma ordem escolhida pelo gerador como se fosse consequência automática da precedência. Para condições, a tradução por saltos pode implementar curto-circuito caso esse seja o contrato adotado.

6. Traduzindo expressões e atribuições

Expressões para máquina de pilha

Em uma máquina de pilha, os operandos são colocados na pilha de operandos; a instrução aritmética remove os operandos necessários e coloca o resultado. A pilha de operandos é distinta da pilha de chamadas: uma guarda valores intermediários de expressões; a outra guarda ativações e estado de chamadas.

```
Expressão MINIC: x = a + b * 2;
  LOAD a
       ; empilha o valor de a LOAD b
       ; empilha o valor de b PUSH 2
       ; empilha o literal 2 MUL
          ; remove b e 2; empilha b*2 ADD
          ; remove a e b*2; empilha a+(b*2)
 STORE x
      ; remove o resultado e grava em x
```

A ordem decorre da estrutura da expressão: multiplicação tem precedência sobre adição; a AST e a IR já devem expressar essa estrutura. Cada instrução deixa a pilha de operandos em estado conhecido. Uma implementação pode ainda especificar tipos e conversões, por exemplo convertendo int para float antes da operação quando exigido pela análise semântica.

Atribuição a elemento de vetor

Para a[i] = valor, o compilador precisa calcular um endereço efetivo, verificar que i é inteiro conforme a regra semântica e respeitar os limites definidos pela implementação. Uma tradução esquemática, supondo elementos de tamanho s e base de vetor base(a), é: endereço = base(a) + i × s; depois STORE_IND indireto. O MINIC exige tratamento de índices e limites, mas o momento e a política exata da verificação de limites devem ser especificados pelo projeto.

```
LOAD i
PUSH tamanho_elemento
MUL PUSH base(a) ADD
 ; endereço efetivo no topo
 ; carregar valor e armazenar indiretamente conforme a convenção da VM
```

7. Condicionais, laços e rótulos

Desvios e rótulos representam fluxo de controle na IR e no código de destino. A tradução de um if separa teste, ramo verdadeiro e, quando houver, ramo falso. O compilador deve assegurar que cada caminho continue no ponto correto e que rótulos sejam únicos dentro do escopo do código gerado.

```
MINIC:
if (x < 10) {
     y = 1;
} else {
     y = 2;
}

3AC:
t1 = x < 10 IF_FALSE t1 GOTO L_else y = 1 GOTO L_end LABEL L_else y = 2 LABEL L_end
```

A condição de if/while/for deve ter tipo bool na versão estrita da especificação. A IR acima mostra primeiro a forma com temporário booleano; um gerador também pode traduzir o teste diretamente para um salto condicional, evitando materializar t1.

Laço while

```
MINIC:
while (i < n) { i = i + 1; }

3AC:
LABEL L_test t1 = i < n IF_FALSE t1 GOTO L_exit t2 = i + 1 i = t2 GOTO L_test LABEL L_exit
```

A condição é testada antes de cada iteração, inclusive antes da primeira. Um for da forma for(init; cond; passo) pode ser reduzido a init seguido de um while com cond e passo no final do corpo. Para continue, o destino deve levar à próxima avaliação/passo correto; para break, ao rótulo de saída. A AST e a IR precisam preservar esses destinos, inclusive em laços aninhados.

8. Funções, prólogo, epílogo e convenção de chamada

Prólogo é a sequência executada na entrada de uma função para estabelecer o frame, reservar espaço local e salvar o estado exigido. Epílogo prepara o resultado, restaura o estado e retorna ao chamador. Os detalhes não são universais: dependem da ABI ou da convenção definida para a VM.

Responsabilidade	Exemplo de decisão da convenção didática
Argumentos	São empilhados da esquerda para a direita (ou passados por registradores); a regra deve ser igual no chamador e no chamado.
Resultado	Funções não void devolvem um valor no topo da pilha de operandos ou em registrador de retorno.
Locais	O prólogo reserva N slots no frame; cada variável tem deslocamento definido pela tabela de símbolos.
Registradores	A convenção identifica quais registradores podem ser sobrescritos e quais devem ser preservados.
Retorno	RET restaura o frame e salta ao endereço de retorno; o chamador recupera e consome o resultado.

O código seguinte é pseudocódigo de uma VM educacional; ENTER, CALL e RET são operações abstratas cujo efeito precisa estar definido pelo interpretador. O exemplo não afirma uma ABI real de processador.

```
; chamada f(a, b) — convenção: argumentos avaliados na ordem documentada
LOAD a
LOAD b
CALL f, 2
; resultado de f fica no topo da pilha de operandos
STORE r
; corpo de f: int f(int x, int y) { int z; ... return z; }
LABEL f
ENTER 1          ; reserva 1 slot local para z …
                 ; x, y e z acessados por slots convencionados
LOAD_LOCAL z LEAVE            ; libera/restaura o frame RET
```

Uma convenção concreta deve resolver se os argumentos ficam na pilha de operandos ou no frame, quem os remove, onde fica o resultado, como são endereçados parâmetros/locais e quem preserva registradores. O exemplo é deliberadamente abstrato para não misturar convenções incompatíveis. Em chamadas recursivas, cada CALL cria um frame novo, inclusive para a mesma função.

9. Máquina de pilha: modelo executável

Uma máquina de pilha didática possui, no mínimo, contador de programa (PC), código de instruções, pilha de operandos e pilha de chamadas/frames. O PC indica a próxima instrução. Operações como ADD consomem operandos e produzem resultado; saltos atualizam o PC; CALL/RET manipulam o controle de execução.

Instrução	Efeito abstrato
PUSH k	Empilha a constante k na pilha de operandos.
LOAD x / STORE x	Lê x e empilha seu valor / remove o topo e grava em x.
ADD, SUB, MUL, DIV, MOD	Consomem dois valores, aplicam operação tipada e empilham o resultado.
CMP_LT	Consome dois valores e empilha um bool.
JUMP L / JUMP_IF_FALSE L	Atualiza o PC para L / consome um bool e salta se for falso.
CALL f, n	Cria ativação de f conforme a convenção e transfere controle.
RETURN / RET	Devolve resultado se necessário, desfaz a ativação e retoma o chamador.

Um verificador de bytecode pode acompanhar a altura da pilha em cada ponto de controle. Em cada junção de fluxo, as alturas esperadas devem ser compatíveis; caso contrário, pode haver bytecode malformado ou tradução incorreta. Tipos dos valores também podem ser verificados se a VM os mantiver explicitamente.

10. Exemplo integrado: função, laço, IR e alvo

Considere um programa MINIC que soma os inteiros de 1 até n. Ele usa declaração local, condição, laço, atualização e retorno, todos contemplados pela especificação.

```
int somaAte(int n) {
     int i = 1;
     int total = 0;
     while (i <= n) {
         total = total + i;
         i = i + 1;
     }
     return total;
}

int main() {
     print(somaAte(4));
     return 0;
}
```

IR de três endereços para somaAte

```
FUNCTION somaAte(n)
i = 1
total = 0
L_test:
t1 = i <= n IF_FALSE t1 GOTO L_exit
t2 = total + i total = t2
t3 = i + 1 i = t3 GOTO L_test
L_exit:
RETURN total
END_FUNCTION
```

A condição aparece antes do corpo; o salto condicional encerra o laço quando i > n. Cada temporário recebe um único resultado. O frame de somaAte mantém n, i, total e temporários que não forem atribuídos a registradores; n pertence à ativação e não é uma variável global.

Trecho de máquina de pilha para o corpo do laço

```
L_test:
LOAD_LOCAL i
LOAD_LOCAL n
CMP_LE JUMP_IF_FALSE L_exit

LOAD_LOCAL total
LOAD_LOCAL i
ADD STORE_LOCAL total
LOAD_LOCAL i
PUSH 1
ADD STORE_LOCAL i
JUMP L_test

L_exit:
LOAD_LOCAL total
RETURN_VALUE
```

LOAD_LOCAL e STORE_LOCAL são operações abstratas com deslocamentos associados aos slots do frame. O prólogo da função cria esses slots; RETURN_VALUE deve transferir o valor ao chamador segundo a convenção. Para n=4, a sequência de totais é 1, 3, 6, 10; a função devolve 10. Esse cálculo serve como conferência semântica da tradução.

11. Erros frequentes de raciocínio

Confusão	Correção
“Toda variável local vive em endereço global.”	Cada ativação precisa de seus próprios valores; locais e parâmetros costumam ser relativos ao frame.
“Recursão usa um frame só porque é a mesma função.”	Cada chamada ativa tem frame próprio; isso preserva parâmetros e estado de retorno.
“Elo de controle e elo de acesso significam a mesma coisa.”	O primeiro acompanha a cadeia dinâmica de chamadas; o segundo, quando necessário, acompanha o ambiente léxico envolvente.
“Heap é obrigatório porque o compilador usa memória.”	Heap é uma região/estratégia para dados dinâmicos; a especificação mínima MINIC não define alocação dinâmica explícita.

Confusão	Correção
“A AST é o código de destino.”	AST é representação do programa-fonte; IR é intermediária; destino é executável na arquitetura ou VM escolhida.
“A ordem de avaliação pode sempre ser trocada para economizar registradores.”	Só se a transformação preservar valor, efeitos, curto-circuito e comportamento de erro definidos pela linguagem.
“Prólogo e epílogo são sequências padronizadas em qualquer máquina.”	São partes de uma convenção específica; devem corresponder à forma de passar argumentos, salvar estado e retornar.

12. Exercícios de revisão

6. Classifique código, uma variável global, um parâmetro, uma variável local e um vetor criado dinamicamente (conceitual) segundo a região mais apropriada no modelo de execução. Justifique pelo tempo de vida, não apenas pelo tipo.
7. Desenhe a pilha de ativações enquanto fatorial(3) executa fatorial(2), que executa fatorial(1). Indique n e o destino de retorno de cada ativação.
8. Explique por que conta pode acessar limite no exemplo da seção 3 sem procurar uma ativação ancestral de função, no MINIC mínimo.
9. Escreva 3AC para if (a != 0) x = b / a; else x = 0; usando rótulos. Mostre que a divisão só ocorre no ramo verdadeiro.
10. Traduza c = a - b para a máquina de pilha definida na seção 6 e indique a altura da pilha de operandos após cada instrução.
11. Liste quatro decisões que uma convenção de chamada precisa padronizar para que CALL e RET funcionem corretamente.

Gabarito comentado e pontos de conferência

1. Código e globais são candidatos a áreas estáticas; parâmetros e locais ordinários pertencem a cada ativação na pilha; um objeto dinamicamente criado com duração que ultrapassa a chamada que o criou é candidato a heap. A especificação MINIC não oferece sintaxe para esse último caso.

2. Frames ativos, do fundo ao topo: main → fatorial(3), n=3 → fatorial(2), n=2 → fatorial(1), n=1. Cada frame guarda retorno para a instrução seguinte à chamada na função-pai.

3. limite é global: sua associação e endereço estático podem ser resolvidos sem percorrer frames. Não há função aninhada capturando local de outra função no MINIC mínimo.

```
t1 = a != 0 IF_FALSE t1 GOTO L_else
t2 = b / a x = t2 GOTO L_end

L_else:
 x = 0 L_end:
```

4. A forma acima garante que a divisão por a está apenas no ramo a != 0; não se deve antecipá-la para antes do teste.

```
LOAD a       ; altura 1
LOAD b       ; altura 2
SUB          ; altura 1, valor a-b
STORE c      ; altura 0
```

5. Convenção precisa definir pelo menos: localização/ordem dos argumentos, local do resultado, registradores preservados e mecanismo de criação/destruição do frame/endereço de retorno. Também deve resolver quem limpa argumentos e como parâmetros/locais são endereçados.

Resumo

A memória estática atende dados de duração global conhecida; a pilha sustenta ativações aninhadas e recursivas; o heap atende dados dinâmicos cuja duração pode não coincidir com uma chamada. O frame preserva os dados e o estado necessários para executar e retornar. Escopo léxico escolhe a declaração; o ambiente de execução fornece o caminho para localizar seu valor.

O gerador traduz IR para um alvo concreto por seleção de instruções, uso de registradores e decisões de avaliação, respeitando tipos, fluxo de controle e convenção de chamada. Uma máquina de pilha é um alvo didático útil, desde que a pilha de operandos não seja confundida com a pilha de chamadas e que o efeito de cada instrução esteja definido.

Referências e alinhamento curricular

AHO, A. V.; LAM, M. S.; SETHI, R.; ULLMAN, J. D. Compiladores: princípios, técnicas e ferramentas. 2. ed. Cap. 7, seções 7.1–7.3; cap. 8, seções 8.1–8.3.

Plano de Ensino da disciplina Compiladores (2026/2), aulas de 30/09, 01/10, 07/10, 08/10, 14/10 e 15/10. A apostila parte da IR e geração semântica iniciadas em 30/09 e 01/10, cobre ambiente de execução e geração de código em 07/10 e 08/10, e deixa blocos básicos, fluxo de dados e otimizações para 14/10 em diante.

Especificação completa da linguagem MINIC, versão 1.0: seções sobre escopo, funções, AST, representação intermediária, geração de código e conformidade. Exemplos desta apostila respeitam a gramática e os tipos nela descritos; escolhas que a especificação deixa em aberto são identificadas como decisões de implementação.